DESARROLLO EN ENTORNO CLIENTE
ALBERTO TURIENZO

PRÁCTICA 2: DEPURAR JAVASCRIPT EN GOOGLE DEVTOOLS

—

Iván López

1. ¿En qué consistía el **bug** original y por qué producía NaN?

   En el bucle se sale del rango del array en la última iteración (intenta iterar en el puesto 3 habiendo 2).

2. Indica **línea y corrección** exacta que resuelve el error.

   ERRÓNEO: for(let i = 1; i <= nums.length; i++){...}

   CORREGIDO: for(let i = 0; i < total + nums[i]; i++){...}

3. Explica la diferencia entre **Step Over**, **Step into** y **Step Out** y cuándo usar cada uno.

   Step Over ejecuta la línea actual y pasa a la siguiente, **Step Into** entra en la función llamada en esa línea, y **Step Out** sale de la función actual y vuelve al nivel superior.

4. ¿Qué información ofrece el panel **Scope** y cómo difiere **Watch**?

   El panel **Scope** permite ver los cambios de cualquier variable disponible dentro del código. Sin embargo, el panel **Watch** muestra la variable de la actual iteración.

5. ¿Qué es el **Call Stack** y cómo te ayudó a entender el flujo de ejecución?

   Call Stack es una función del DevTools que muestra, de manera ordenada, las funciones ejecutadas hasta el punto de ejecución actual.

   Me ayudó a comprender el flujo de ejecución mostrando las funciones ejecutadas en su orden original de ejecución.

6. ¿Cómo se crea un **breakpoint condicional** y para qué casos es útil? Pon el condicionante que usaste.

   Un **breakpoint condicional** se crea haciendo clic derecho en la sección de breakpoint, clicando en "crear breakpoint condicional" e indicando la condición que debe cumplirse para parar la ejecución.

   Es útil cuando intuyes que el programa se rompe en una iteración de un bucle.

   El condicionante utilizado en la práctica fue: **i === nums.length**.

7. ¿Qué son los **Event Listener Breakpoints** y cómo los aplicaste en esta práctica?

   Los **Event Listener Breakpoints** permiten pausar la ejecución del programa disparando un evento en concreto, como *click* o *keydown*.

8. Compara **console.log** vs **log points**: ¿ventajas de estos últimos durante la depuración?

   El **console.log** se indica en el propio código fuente del programa. Sin embargo, los **log points** se pueden ejecutar sin tener que modificar ni detener el código. Esto da una ventaja muy grande a la hora de depurar, evitando tener que modificar el código fuente.

9. ¿Qué ocurre si activas **Pause on exceptions**? Describe un caso en tu código donde se dispare.

   Al activar **Pause on exceptions**, el depurador detiene la ejecución del código en cuanto aparece una excepción.

   En el código, se dispara una excepción a la hora de intentar iterar por última vez sobre el array de números.

10. Tras separar a script.js, ¿qué ventaja aporta usar defer en la etiqueta <script>?

    Ofrece la ventaja de tener dos scripts distintos ejecutándose en el mismo html. Permitiendo dividir el programa.

11. Menciona dos formas de **inspeccionar/alterar variables** durante una pausa de depuración.

    Las dos opciones más conocidas son: en la consola de depuración general, y en la consola de depuración desde la vista del script. La segunda opción permite añadir breakpoints a medida que vas depurando el programa.

12. Si el array de números está vacío, ¿qué resultado debe mostrar la interfaz y por qué?

    Si el array está vacío, debe mostrar **NaN**. Es lo indicado en el programa, dentro de la función calcularMedia(nums).

13. Diseña una **regla de prueba** (valores de entrada y resultado esperado) que verifique que el bug ha sido corregido.

    Valor de Entrada: 10 Resultado Esperado: 1

14. ¿En qué situaciones preferirías un **log point** frente a detener la ejecución con un breakpoint?

    Esa opción sería elegible durante la primera ejecución del programa, para poder ver cómo evolucionan las variables.

15. (Opcional) Explica cómo investigarías un **bug intermitente** asociado a eventos rápidos de usuario (p.ej., doble clic).

    Añadiendo **Event Listener Breakpoints**. De esa manera, puedo simular la interacción del usuario ¡.